

1. Why do most developers lean toward constructor injection rather than other forms of dependency injection?

Developers prefer constructor injection because it ensures that the object is created in a **valid state**, as all necessary dependencies are guaranteed to be provided. It also makes dependencies clearly visible and helps achieve **immutability** (by using final fields), which naturally promotes thread safety and cleaner code.

2. In which scenarios would a singleton bean be the right choice, and when would using a prototype bean be more suitable?

A **singleton** bean is the right choice for **stateless** components like services, repositories, and utility classes, as they can be safely shared across the application. A **prototype** bean is more suitable for **stateful** components (like session counters or custom data holders) where each user or process requires its own fresh, unique instance to avoid concurrency issues.

3. Does the "singleton" scope in Spring guarantee thread safety? Explain the reasoning clearly.

No, the "singleton" scope **does not** guarantee thread safety. Spring only ensures that there is only **one instance** of the bean in the container. If that single instance contains mutable state (variables that change), multiple threads accessing and modifying it concurrently will lead to race conditions. Thread safety must be implemented manually by the developer (e.g., using synchronized).

4. Is it possible to split your Spring configuration across multiple classes? How does the framework combine them internally?

Yes, it is common practice to split configuration across multiple classes for better organization. Spring combines them internally using the **@Import** annotation. A main configuration class can import other specific configuration classes (like SecurityConfig or DatabaseConfig), and the framework treats all of them as a single set of instructions for the container.

5. What special behavior does Spring apply to beans that are defined with the prototype scope throughout their lifecycle?

Spring manages the creation and basic injection of prototype beans but stops managing them immediately after creation. Spring **does not** track the prototype bean's lifecycle and will **not** call methods like **@PreDestroy**. Once the prototype bean is delivered to the requesting component, the client is responsible for its cleanup.

6. What are Spring Profiles used for, and how can you switch from one profile to another during runtime or startup?

Spring Profiles are used to define environment-specific configurations (e.g., dev, test, prod). This allows Spring to load only the relevant beans for a specific environment (e.g., loading an H2 database bean in dev and a MySQL bean in prod). You can switch profiles at startup using a command line argument like `--spring.profiles.active=prod`.

7. What is the functional difference between **BeanFactory** and **ApplicationContext**, and when would each be appropriate?

The **BeanFactory** is the most basic container, providing simple IoC and DI functionality, and is often used in resource-constrained environments. The **ApplicationContext** is the advanced version; it includes all **BeanFactory** features plus enterprise features like component scanning, event publishing, internationalization, and transaction management. **ApplicationContext** is almost always used in modern development.

8. Under what circumstances would using a **BeanFactory** make more sense than using an **ApplicationContext**?

Using a **BeanFactory** makes sense only in extreme, resource-constrained environments, such as embedding Spring within a mobile device or a specialized Java system with minimal memory. This is because **BeanFactory** initializes resources lazily (on demand) and avoids loading the extra enterprise features provided by the **ApplicationContext**.

9. What causes a circular dependency in Spring, and what exactly does the term mean in practical terms?

A **circular dependency** means that two or more beans rely on each other to be created (e.g., Bean A needs Bean B injected, and Bean B needs Bean A injected). In practical terms, the container gets stuck in an infinite loop trying to initialize both objects simultaneously, making it impossible to satisfy the dependencies.

10. How do **@Primary** and **@Qualifier** differ when Spring has multiple candidates for the same type?

When multiple beans of the same type exist, **@Primary** marks one bean as the highest priority default choice for autowiring. **@Qualifier** provides a distinct name to explicitly specify which exact bean should be injected, overriding both the default and the **@Primary** choice.

11. How can Spring read values from system properties or environment variables and inject them into beans?

Spring uses the **@Value** annotation, which takes a SpEL (Spring Expression Language) expression to read external properties. You can inject values directly from the environment using the syntax `@Value("${variable.name}")`. Spring's **Environment** abstraction handles

resolving the value from .properties files, command-line arguments, or OS environment variables.

12. Between XML configuration and annotations, which approach is more widely used today, and what's the reason for that preference?

Annotations (@Component, @Autowired) are overwhelmingly more widely used today than XML configuration. The reason is that annotations are placed directly in the Java source code, making the configuration context-aware, more compact, and easier to maintain than managing separate, verbose XML files.

13. How is JpaRepository different from CrudRepository in terms of capabilities?

CrudRepository provides only basic CRUD (Create, Read, Update, Delete) operations. **JpaRepository** extends CrudRepository and adds more advanced capabilities specific to JPA (Java Persistence API), such as bulk operations, pagination, sorting, and flushing changes to the database. JpaRepository is the preferred choice for complex applications.

14. What does the @Transactional annotation actually achieve inside a Spring application?

The **@Transactional** annotation tells Spring to wrap the method execution within a database transaction. Spring uses a **Proxy** around your bean to intercept the method call; before the method runs, it opens a connection and transaction, and upon successful return, it commits the transaction (or rolls back if an exception occurs).

15. When writing tests with Mockito, what separates @Mock from @Spy in behavior and purpose?

@Mock creates a completely fake object that has no original code logic; every method call must be manually instructed to return a value. **@Spy** creates a partial, real object that keeps all the original class logic, but allows you to selectively override (mock) specific methods if needed.

16. When you use @Autowired, what selection strategy does Spring apply to decide which bean should be injected?

Spring applies a three-step selection strategy:

1. **By Type:** It tries to find a single bean that matches the dependency type.
2. **By Name:** If multiple beans match the type, it looks for a bean whose name matches the dependency variable name.

3. **By Qualifier/Primary:** If ambiguity remains, it uses the explicit **@Qualifier** name or the implicitly designated **@Primary** bean.

17. How do **@PostConstruct** and **@PreDestroy** influence the initialization and shutdown phases of a bean's lifecycle?

@PostConstruct marks a method that Spring must execute exactly once, immediately after the bean is created and all dependencies are injected. **@PreDestroy** marks a method that Spring must execute exactly once, just before the container shuts down or removes the bean instance from memory.

18. What is the point of using **@Lazy**, and in what situations would delaying bean creation be helpful?

The point of using **@Lazy** is to delay the creation of a bean until it is actually needed for the first time. This is helpful for complex setup or resource-heavy beans that are rarely used, or to break specific types of circular dependencies during application startup, improving the overall startup time.

19. How does Spring's component scanning mechanism detect components, and how can you control or narrow down the scanning area?

Spring's component scanning mechanism searches specified base packages for classes annotated with stereotype annotations (like **@Component**, **@Service**, **@Repository**). You can narrow the scanning area by defining the base package explicitly in **@ComponentScan(basePackages = "com.myapp.services")**. This speeds up startup and avoids loading unnecessary classes.